

Assignment #02

FSStatLib

Design and implementation report

Academic year 2025-2026 - Concurrent and Distributed Programming

This report documents the submitted implementation and its concurrency choices. The design follows the terminology and principles in merged.pdf and the assignment README.

1. Problem analysis

getFSReport recursively enumerates a directory, counts regular files and classifies each size into NB equal bands over $[0, \text{MaxFS}]$, plus one overflow band. Directory depth, filesystem latency and the number of entries are not known in advance. The method must therefore return before the scan completes and surface failures asynchronously.

Band i contains sizes for which $\text{floor}(\text{size} * \text{NB} / \text{MaxFS})$ equals i , except that MaxFS belongs to the final in-range band. Values greater than MaxFS belong to index NB. Every result contains exactly $\text{NB} + 1$ counters and their sum equals totalFiles .

2. Three deliberately separate designs

Version	Public result	Discipline
Event loop	Vert.x Future<FSReport>	readDir and props are non-blocking; handlers only compose futures
Reactive	RxJava Single<FSReport>	A cold Flowable recursively emits files and reduce builds the report
Virtual threads	CompletableFuture<FSReport>	Natural blocking Files API; one cheap virtual task per file size

The versions intentionally do not share traversal or report code. This preserves the programming discipline requested by the assignment: callbacks/futures remain visible in the event-loop implementation, operators define the Rx data flow, and the virtual-thread version keeps direct sequential control flow.

3. Correctness and failures

- Parameters are validated before traversal; MaxFS and NB must be positive.
- Empty directories complete with total zero and $\text{NB} + 1$ zero counters.
- Filesystem failures terminate the Future, Single or CompletableFuture exceptionally.
- Rx closes every Files.list stream through Flowable.using; the virtual-thread version closes Files.walk and its executor with try-with-resources.
- The event-loop handlers perform no blocking calls, preserving event-loop responsiveness.

4. Verification

A shared black-box JUnit scenario creates an empty file, a 10-byte file in a subdirectory and a 21-byte file. With MaxFS 20 and two bands, all implementations must return $\text{totalFiles} = 3$ and distribution $[1, 1, 1]$. The test also enforces bounded completion times for the two future-based versions. Run mvn test from assignment-02.